

A Small Study of Skills in Local Coding Agents

Jiacheng Chen

National University of Singapore

9 October 2026

Abstract

This study looks for a task that a smaller local coding model can complete with a skill, while a larger model can complete it both with and without the skill. The task checks dialogue graphs and exports valid graphs as JSON and SVG. In the final batch, Qwen3.5 9B with a public graph skill and Qwen3.5 27B in both conditions pass all ten functional checks. Both supplied-skill programs use the original validator and renderer. The 27B run with skill takes longer than its baseline in this batch. These results describe three individual runs after task selection; they do not show that the skill improves the smaller model or generalizes to other tasks.

1 Introduction

In this study, a skill is a package containing instructions and supporting code that a coding agent can read and use. I wanted to find one task with three successful conditions: a smaller candidate model with the skill, and a larger candidate model both with and without it. I also compared their commands, generated programs and running time.

The final candidates are local Qwen3.5 9B and 27B models, run through the same Codex command-line harness. The size labels are used to select the candidates; this study does not independently establish their relative ability across tasks. There is no 9B without-skill run in the final batch.

2 Task and Skill

The input contains six JSON dialogue graphs. A graph has nodes with IDs, text, speakers and types, and directed edges with source, target and text fields. Three graphs are valid: a branching dialogue with 12 nodes and 17 edges, a loop, and a single terminal transition. Three others have missing edge endpoints.

The agent writes `solution.py` to produce an audit for every graph, including its validity, endpoint errors, and node and edge counts. It also

exports each valid graph as JSON and a real Graphviz SVG. The JSON must retain the input contents. The SVG must retain the node IDs, directed connections, and box or diamond shapes for the two node types. The program must expose `audit_graph(data)` for additional inputs.

The endpoint rule has one exception: `End` may be a missing target, but it is not exempt as a source. For example, an edge from a missing `Ghost` to a missing `Absent` must report both errors. An edge from a missing `End` to an existing node must still report the missing source. Cycles are allowed.

The unchanged `dialogue-graph` skill comes from SkillsBench [1]. Its Python helper provides `Graph.from_dict`, `validate`, `to_json` and `visualize`. It supports graph operations but does not provide a dialogue-text parser. The task here was designed for this study; its score is not a SkillsBench benchmark result.

3 Experiment Setup

3.1 Runs and checks

The final batch runs 9B with skill first, then 27B without skill, then 27B with skill. Each has a new workspace and session, with a 360-second limit for the model turn, including loading. All three receive the same prompt, graph inputs, checker and shell tools. The supplied-skill conditions also receive the original package and its import path. The shared prompt asks the agent to reuse the four methods if the package is present.

Ten functional checks cover the audit format, endpoint diagnostics, graph counts, JSON contents, SVG XML and graph structure, API behavior, and input preservation. The API check includes the six supplied graphs and two additional inputs. These checks cover parts of one task and are not ten independent tasks.

I also check normal completion and unchanged input and skill files. For the supplied-skill runs, I inspect whether the agent reads the skill and calls its original helper. This is separate from check-

ing whether it follows every requested method. In particular, a correct JSON export can pass the functional check even if it does not use `to_json`.

The saved programs and outputs were rechecked without editing them. To observe helper calls, an unchanged copy of each program was executed with method-call recording. This records how the program works; it is not another model trial or a repaired answer.

3.2 Local configuration

The experiments ran on a MacBook Pro with M5 Max, 18 CPU cores, 40 GPU cores and 64 GB memory. The recorded software versions are macOS 27.0.1, Codex 0.161.0, Ollama 0.40.1, Python 3.14.6 and Graphviz 16.1.0.

Model metadata reports 9.7B and 27.4B parameters, MLX/NVFP4, and a loaded context of 65,536. Both use defaults including temperature 1, top-p 0.95 and top-k 20; no random seed is fixed. Only 9B lists `draft_num_predict=3`. This difference, model loading and the fixed run order limit comparisons of speed. Exact model digests and run-time snapshots are included in the saved record [2].

4 Results

Condition	Checks	Time (s)	Cmds.
9B + skill	10/10	39.033	21 (1)
27B no skill	10/10	126.039	10 (2)
27B + skill	10/10	178.625	15 (1)

Table 1: Final local runs. Command counts include completed shell events; parentheses count nonzero exits. Time includes model loading, but excludes preparation and independent checking.

All three configurations pass 10/10 checks and finish normally (Table 1). The input and supplied skill files remain unchanged. Each agent writes its complete solution once in the visible trace. Both supplied-skill programs execute the original validator and renderer. The required three-success case is therefore observed in this batch.

4.1 What the programs do

9B with skill. The agent reads the skill and helper, checks imports, writes the program, and inspects the outputs. Its API loads data with `from_dict` and calls `validate`; its SVG export uses `visualize`. Batch reexecution records six validations and three renders on the supplied graphs. All eight API inputs also call the orig-

inal validator. JSON is exported with Python’s `json.dump`, rather than the requested `to_json`. Thus it completes the functional task and uses the helper, but does not follow the full four-method request.

27B without skill. After checking for the absent package, the agent implements endpoint validation, JSON export and DOT construction itself. It runs `dot -Tsvg` to render the graphs. Both the output checks and additional API checks pass.

27B with skill. The program uses all four requested methods. Batch reexecution records twelve validations, three JSON exports and three renders. It validates all six graphs again during export, accounting for the extra validation calls. All eight API inputs also use the original validator.

The nonzero shell commands are file or output probes: 9B reads a nonexistent `__init__.py`; the baseline checks absent skill paths; 27B with skill searches for literal `doublecircle`, which is not how the SVG represents the shape. There are also recoverable tool-router errors outside the shell counts: an unavailable image tool in the baseline, and file, process-ID and persistent-thread tool errors in 27B with skill. These did not prevent completion, but normal completion does not imply that every tool call succeeded.

4.2 Time and token counts

The 27B run with skill takes 52.586 seconds and five commands more than the run without it. It includes more package inspection and repeats graph validation. The 9B run is faster in wall time despite making more commands. With one run per condition and different model settings, I cannot attribute these differences to the skill alone.

Condition	Input	Cached	Output
9B + skill	277,160	253,898	3,411
27B no skill	113,361	98,376	3,012
27B + skill	238,591	219,201	3,921

Table 2: Completed-turn token counters, accumulated across requests. Cached input is included in input and should not be added to it. These local counters are not monetary API costs.

Table 2 reports the usage counters returned by Codex. They sum multiple model requests, so input totals can exceed the loaded context size. Returned reasoning-token counts are zero; this does not establish the model’s internal behavior.

5 Earlier Attempts

The final case was selected after earlier failures. The first task parsed dialogue text, which required an algorithm the skill did not supply. Early attempts also encountered authentication, checker and sandbox temporary-file problems. Later 27B baseline attempts passed 34/35 and 30/35 checks under different revisions. These results are kept separately.

A smaller parsing task then gave 27B scores of 9/10 without skill and 10/10 with skill, while both 4B runs timed out. Neither supplied-skill run showed a skill read. An explicit parsing screen gave 4B with skill 2/10 and stopped the other conditions under its planned gate.

I next changed the task to graph auditing and export, matching the helper’s existing functions. In the first graph version, 4B with skill passed the functional checks, but an extra requirement about where helper calls occurred stopped the schedule. That restriction went beyond the study’s original goal and was removed before new runs; the original failed record was retained.

In graph V2, the 27B conditions passed, while 4B with skill scored 8/10 by saving DOT text as SVG. The generated supplied-skill programs did not execute the helper. In V3, the prompt made method reuse explicit. The 27B conditions passed again, while 4B with skill scored 9/10: its SVGs were valid XML but lacked the required graph contents.

The final batch changes the smaller candidate to 9B and reruns all three conditions with the V3 task, inputs and checker unchanged. It does not combine an old smaller-model result with newer larger-model runs. Earlier outputs and the full search summary remain in the submission archive.

6 Discussion and Limits

The useful distinction in these attempts is between reading a skill and using it correctly. A file named SVG, or an import of the helper, was not sufficient. Inspecting the graph contents and actual method calls helped identify the failed attempts and explain the final programs.

The final batch answers the narrow case-finding question: the selected smaller model with skill and larger model in both conditions complete the same task. It does not show that the skill caused 9B to succeed. There is no final 9B baseline, each condition runs once, and both the task and smaller candidate were selected after earlier attempts. The skill also contains executable code, so any benefit cannot be assigned to its instructions alone. The checks cover a finite set of inputs.

A useful next step would be to add 9B without skill and repeat the four conditions with a fixed task and varied run order. A timing study would also need matching backend settings and separate loading measurements. These extensions have not been performed here.

7 Reproduction

Batch 20261009-183344-d622 is the final run record. The initial 9B download took 370.282 seconds and is excluded from the model-turn time. The submission includes all seven raw exports, original programs and outputs, event logs, source files and offline rechecking scripts. No generated solution was repaired to obtain a passing score.

Reproduction code, task files, the checker and the final raw archive are provided with this report. The README gives commands for a new local run and for offline rechecking. The source commit, model digests, earlier attempts and tooling assistance are recorded in `PROVENANCE.md`. The code has been reorganized for reproduction; the exact version used for these results remains in the raw archive.

References

- [1] benchflow-ai and scriptwonder. SkillsBench, dialogue-graph skill. Apache-2.0. Revision 55bfe693f2a1; full commit recorded in the project. <https://github.com/benchflow-ai/skillsbench>.
- [2] Jiacheng Chen. 2026. Local graph-audit experiment record, 9 October 2026. Unpublished run archive, batch 20261009-183344-d622, supplied with this report.